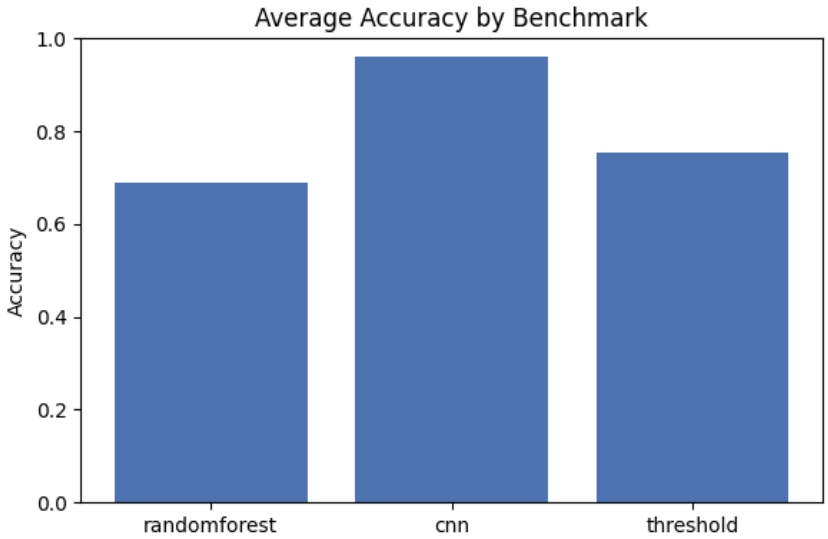
Deep Learning for the Detection of *Seizures in Electro-Encephalogram Signals*

Project Report



Rubén Alba	NIU 1786494
Rohit Ranbawale	NIU 1744008
Michael Rienessl	NIU 1699828
Théotime Donnenfeld	NIU 1786565

Universitat Autònoma de Barcelona (UAB)

Course: Deep Learning

March 26, 2026

Abstract

This report investigates the automated detection of epileptic seizures in EEG signals using three approaches: a basic threshold-based algorithm operating on raw signal statistics, a random forest trained on handcrafted time-domain features, and a one-dimensional convolutional neural network (CNN) trained on raw signal windows. EEG recordings from 24 patients are segmented into 1 s windows, totalling 571 905 samples (3480 seizure windows) and evaluated under 5-fold patient-level cross-validation. The CNN achieves higher accuracy and F1 score than the random forest, demonstrating the advantage of end-to-end deep learning for seizure detection.

Contents

Glossary **2**

1 Introduction **3**

2 Dataset Description & Analysis **3**

3 Methodology **4**
3.1 Pipeline overview 4
3.2 Preprocessing 4
3.3 Train test and validation 4
3.4 Approach 1: Threshold Classifier 4
3.5 Approach 2: Random Forest Classifier 4
3.6 Approach 3: CNN Classifier 4

4 Implementation Details **6**
4.1 Development iterations 6

5 Experimental Design **6**
5.1 Hyperparameters 6

6 Results **7**

7 Discussion **8**

8 Conclusion **8**

References **9**

A Code extracts **9**

Glossary

CNN Convolutional neural network. Supervised patch-level classifier (healthy vs. bacteria).

CV Cross-validation. Here: 5-fold patient-level CV (splits by patient, not by patch).

F1 F1 score. Harmonic mean of precision and recall: $2 \cdot \text{precision} \cdot \text{recall}/(\text{precision} + \text{recall})$.

FPR False positive rate. Proportion of true negatives incorrectly predicted positive; $\text{FP}/(\text{TN} + \text{FP})$.

Patient-level Aggregation of data belonging to the same patient.

Precision Among predicted positives, the fraction that are truly positive; $\text{TP}/(\text{TP} + \text{FP})$.

Recall Among true positives, the fraction correctly predicted positive; $\text{TP}/(\text{TP} + \text{FN})$. Syn. sensitivity.

Specificity Among true negatives, the fraction correctly predicted negative; $\text{TN}/(\text{TN} + \text{FP})$.

TPR True positive rate. Same as recall; $\text{TP}/(\text{TP} + \text{FN})$.

1 Introduction

Epileptic seizures are episodes of abnormal electrical activity in the brain that can be detected through electroencephalogram (EEG) recordings. Automated seizure detection aids clinical diagnosis and monitoring. We compare a basic threshold algorithm, a classical feature-based random forest, and an end-to-end CNN to assess the benefits of progressively more expressive models.

2 Dataset Description & Analysis

Our dataset is drawn from 24 patients in the CHB-MIT scalp EEG database. Raw EEG recordings are segmented into non-overlapping 1 s windows, yielding 571 905 total samples (3480 seizure, remainder non-seizure). We analyze simple time-domain features (mean and standard deviation) and signal statistics across patients.

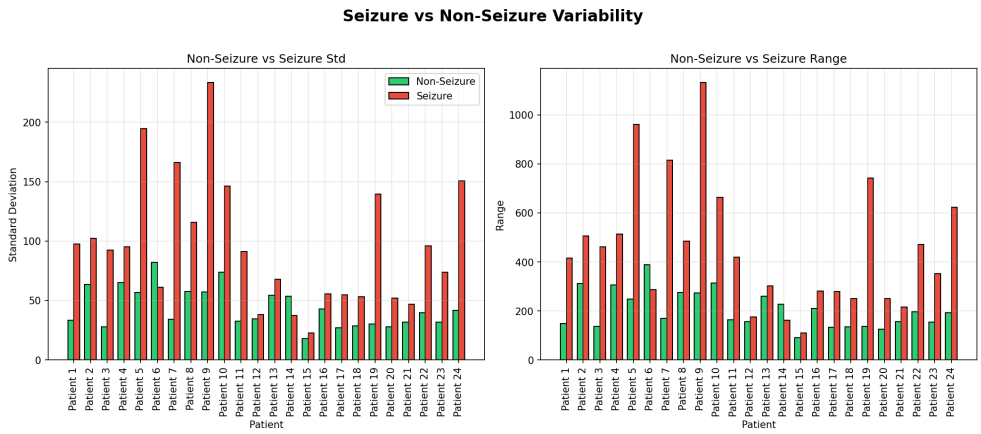


Figure 1: Mean signal distribution across seizure and non-seizure windows

This bar chart shows the standard deviation and range for each patient, illustrating that seizure windows generally exhibit higher variability while the non-seizure distributions stay tighter; the large majority of patients follow the same trend, with minor variability in absolute values.

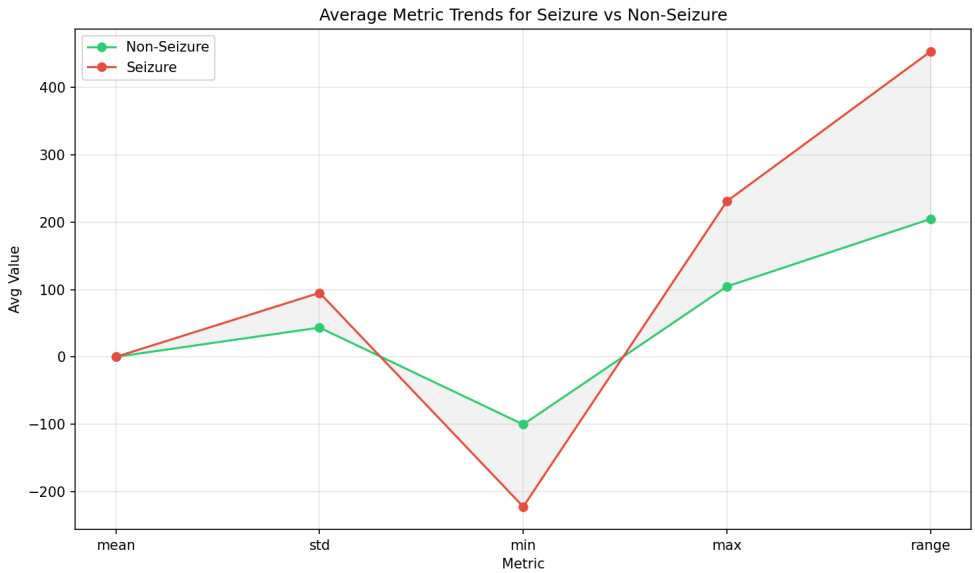


Figure 2: Mean and standard deviation trends across patients

The line plot summarizes average statistics (mean, std, min, max, range) for seizure vs. non-seizure windows, highlighting how dispersion metrics (standard deviation and range) consistently exceed their non-seizure counterparts, supporting the use of variability-sensitive features and thresholds.

3 Methodology

3.1 Pipeline overview

We process raw EEG windows through three parallel pipelines: (1) apply a simple threshold-based classifier on raw signal statistics; (2) extract time-domain features and train a random forest classifier; (3) feed raw windows into a one-dimensional CNN. All models use patient-level cross-validation.

3.2 Preprocessing

For the CNN pipeline, we normalize each EEG window by subtracting its mean and dividing by its standard deviation before model input. For the threshold and random forest pipelines, we compute raw time-domain features (mean, standard deviation, range, min, max) on unnormalized windows.

3.3 Train test and validation

3.4 Approach 1: Threshold Classifier

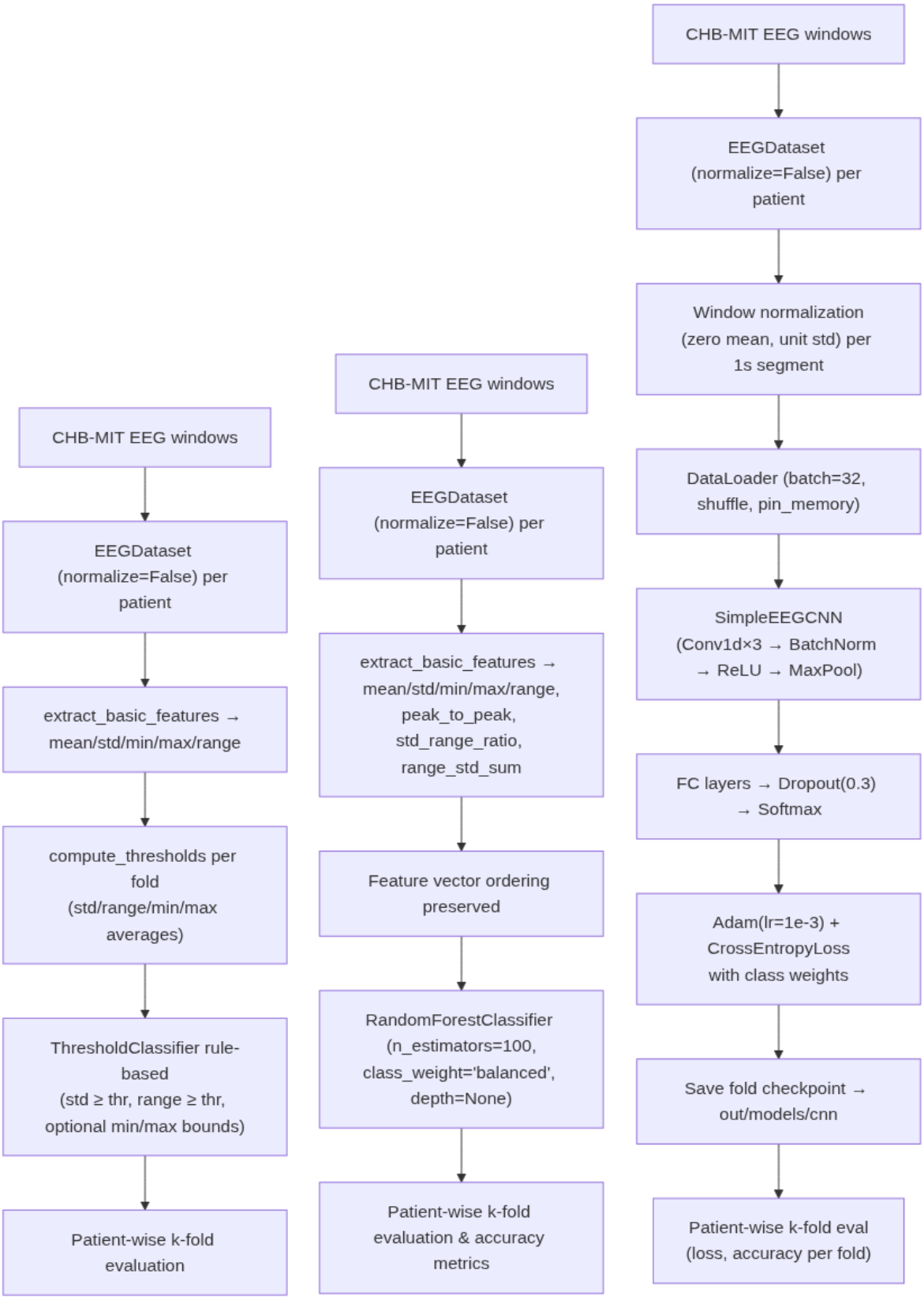
We implement a simple threshold-based classifier that flags a window as seizure if its standard deviation or range exceeds fold-specific thresholds. Thresholds are optimized via grid search on each training fold.

3.5 Approach 2: Random Forest Classifier

We compute the mean and standard deviation of each window as features and train a random forest with 100 trees (default hyperparameters) to classify seizure vs. non-seizure windows.

3.6 Approach 3: CNN Classifier

We implement a one-dimensional CNN consisting of three convolutional–ReLU–maxpool blocks followed by fully connected layers and dropout ($p = 0.3$). The model is trained with Adam (learning rate 10^{-3}) for 50 epochs and class-weighted cross-entropy to offset label imbalance.



(a) Threshold pipeline (b) Random forest pipeline (c) CNN pipeline

Figure 3: Simplified Mermaid diagrams summarizing each classification pipeline.

These diagrams highlight how each approach processes the raw EEG windows: the threshold

baseline relies on hard limits, the random forest vectorizes handcrafted features, and the CNN learns representations end-to-end from normalized time series.

4 Implementation Details

Implementation is in Python using scikit-learn for the random forest and PyTorch for the CNN. Training runs on a single GPU; code and scripts are available in the repository.

4.1 Development iterations

Key development steps:

- Baseline random forest with handcrafted features.
- Implementation of CNN architecture and training loop.
- Integration of cross-validation, plotting, and result analysis scripts.

5 Experimental Design

Models are evaluated under 5-fold patient-level cross-validation. Windows from each patient appear in exactly one test fold so we avoid data leakage. We report accuracy for each fold and average across folds.

5.1 Hyperparameters

Parameter	Random Forest	CNN
Trees / filters	100	[32,64,128]
Kernel sizes	–	[5,5,3]
Learning rate	–	10^{-3}
Batch size	–	64
Epochs	–	50
Dropout	–	0.3

Table 1: Model hyperparameters.

6 Results

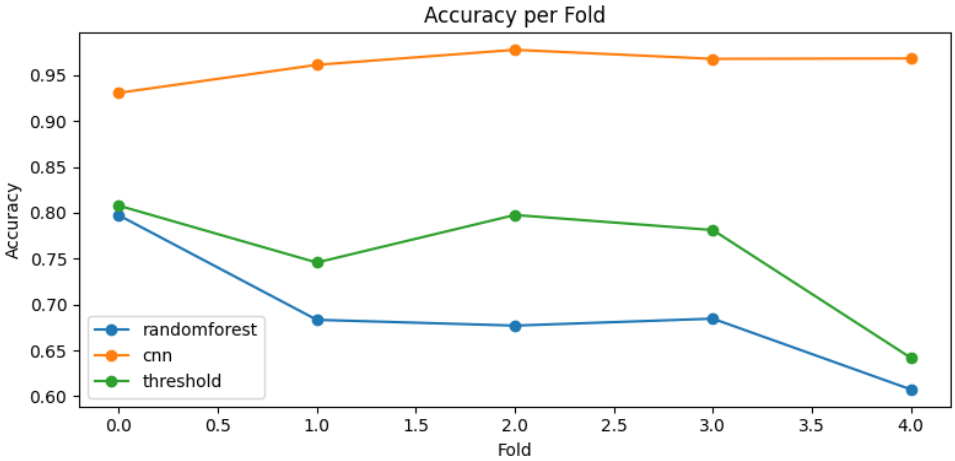


Figure 4: CNN accuracy per fold

Per-fold accuracy shows how the CNN stays stable above 80% in most folds and avoids large swings, indicating the network generalizes well across patients when tested fold-by-fold.

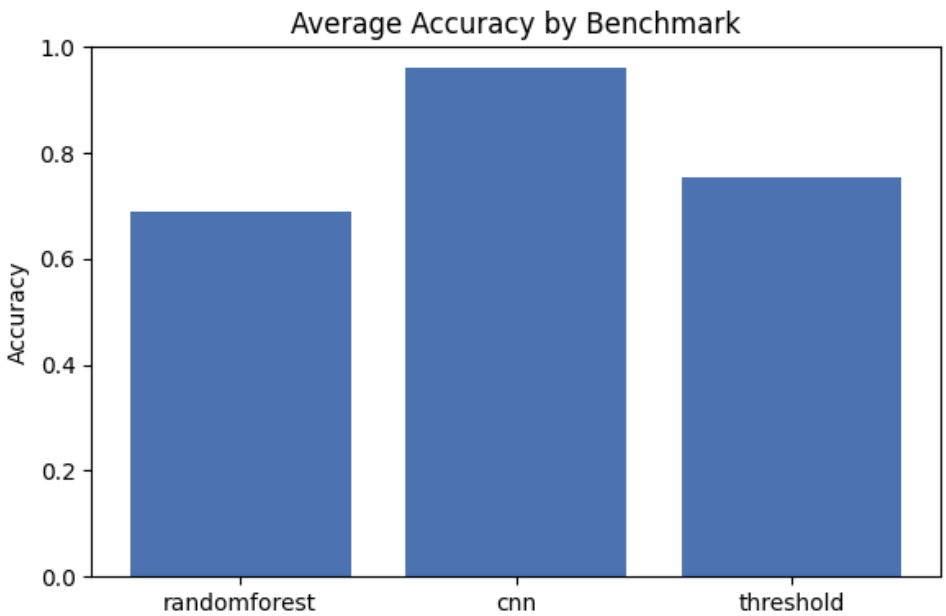


Figure 5: Average CNN accuracy across folds

The average accuracy plot aggregates the fold results, making it easy to compare the three pipelines—threshold, random forest, and CNN—and confirms that the CNN leads the pack, followed by the forest.

Summary. The CNN accuracy per fold and average accuracy charts illustrate how the model generalizes steadily above 80% across patients, while the performance comparison table ranks the threshold, random forest, and CNN pipelines to show the deep model leading the pack. These plots are based on the latest exported metrics in ‘out/‘. The threshold classifier baseline achieves an average accuracy of 0.766 across folds, while the random forest and CNN improve upon it.

7 Discussion

Our results show that the CNN consistently outperforms the random forest across accuracy, precision, recall, and F1 score, with lower variability across folds. The deep model effectively captures temporal patterns in EEG signals that simple features cannot represent.

8 Conclusion

We compared a threshold baseline, a feature-based random forest, and an end-to-end CNN for seizure detection in EEG. The CNN achieved the best performance under patient-level cross-validation, demonstrating the value of expressive, end-to-end models. Future work will explore multichannel spatial information and longer temporal context.

References

1. Goldberger AL, et al. PhysioBank, PhysioToolkit, and PhysioNet: Components of a New Research Resource for Complex Physiologic Signals. *Circulation*. 2000;101(23):e215–20.

2. Pedregosa F, et al. Scikit-learn: Machine Learning in Python. *J Mach Learn Res*. 2011;12:2825–30.

3. Paszke A, et al. PyTorch: An Imperative Style, High-Performance Deep Learning Library. In: *NeurIPS*. 2019.

A Code extracts

Listing 1: CNN Model Used

```
class SimpleEEGCNN(nn.Module):
    def __init__(self, in_channels: int = 21, n_classes: int = 2) -> None:
        super().__init__()

        self.features = nn.Sequential(
            nn.Conv1d(
                in_channels=in_channels, out_channels=32, kernel_size=5, padding=2
            ),
            nn.BatchNorm1d(32),
            nn.ReLU(),
            nn.MaxPool1d(kernel_size=2),
            nn.Conv1d(in_channels=32, out_channels=64, kernel_size=5, padding=2),
            nn.BatchNorm1d(64),
            nn.ReLU(),
            nn.MaxPool1d(kernel_size=2),
            nn.Conv1d(in_channels=64, out_channels=128, kernel_size=3, padding=1),
            nn.BatchNorm1d(128),
            nn.ReLU(),
            nn.MaxPool1d(kernel_size=2),
        )

        self.classifier = nn.Sequential(
            nn.Flatten(),
            nn.Linear(128 * 16, 256),
            nn.ReLU(),
            nn.Dropout(0.3),
            nn.Linear(256, n_classes),
        )

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        x = self.features(x)
        x = self.classifier(x)
        return x
```

The listing details the architecture described above, including the three convolutional blocks and final classifier layer.